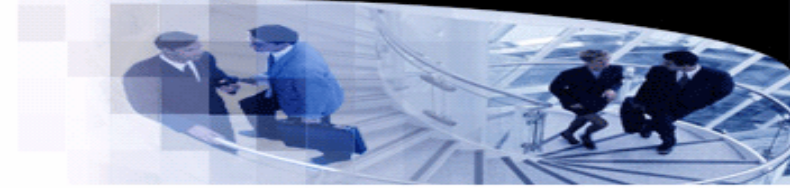




武汉大学
WUHAN UNIVERSITY



Programming Languages and Compilers

Lecture 16. Attribute Grammars

袁梦霆 (YUAN Mengting)

ymt@whu.edu.cn

School of Computer Science, Wuhan University

Semantics

- Introduction

- Some programming languages torture their user by undefined behaviors.

7.12.1 Sequence Points

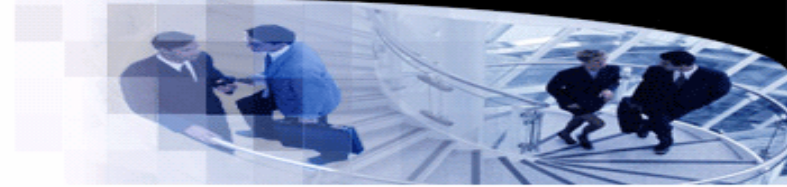
In Standard C, if a single object is modified more than once between successive sequence points, the result is undefined. A *sequence point* is a point in the program's execution sequence at which all previous side effects of execution are to have taken place and at which no subsequent side effects will have occurred. Sequence points occur:

- at the end of a full expression—that is, an initializer, an expression statement, the expression in a **return** statement, and the control expressions in a conditional, iterative, or **switch** statement (including each expression in a **for** statement)
- after the first operand of a **&&**, **| |**, **? :**, or comma operator
- after the evaluation of the arguments and function expression in a function call

According to this rule, the value of the expression `++i*++i` is undefined as is the prior

"C, A Reference Manual"

- Rigorous semantics are therefore required.



Semantics

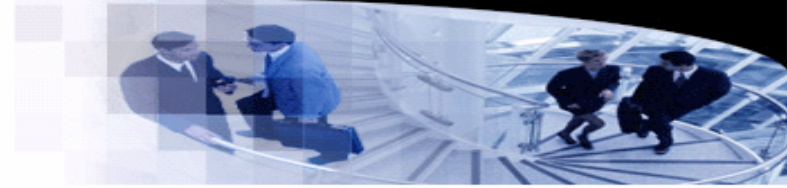
• Introduction

- Semantics define the evaluations of expressions (programs).
- Operational semantics are over detailed.

$\rightarrow$ (untyped)			
Syntax		Evaluation	
$t ::=$			$t \rightarrow t'$
x	terms:		
$\lambda x. t$	variable	$\frac{t_1 \rightarrow t'_1}{t_1 t_2 \rightarrow t'_1 t_2}$	(E-APP1)
$t t$	abstraction		
	application	$\frac{t_2 \rightarrow t'_2}{v_1 t_2 \rightarrow v_1 t'_2}$	(E-APP2)
$v ::=$	values:		
$\lambda x. t$	abstraction value	$(\lambda x. t_{12}) v_2 \rightarrow [x \mapsto v_2] t_{12}$	(E-APPABS)

- Compilers need an abstract definition of semantics, as compilers do not care about running details.

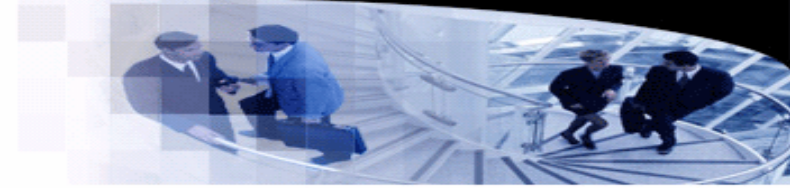
Attribute Grammars



- Introduction

- Compilers often resort to attribute grammars.
- An attribute grammar can be obtained by adding **attributes** and **evaluation rules** to a grammar.
- Attribute grammars are tightly related to grammars.
- It is easy to integrate attribute grammars with parsers.
- What are the meanings of the following statements?
 - $1 + 2 * 3;$
 - `int p, q = 2;`
 - `*p = q + 2;`

Attribute Grammars



- **Definition**

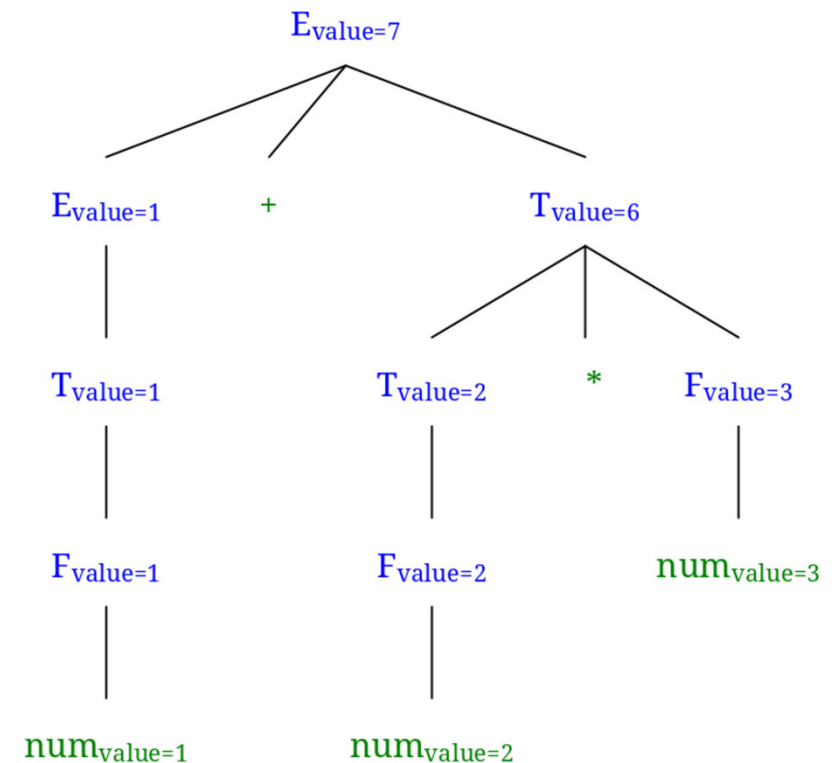
- A set of attributes $A(X)$ for each symbol X .
- A set of evaluation rules $E(p)$ for each production p .
- Each evaluation rule has the form $x = f(y_1, y_2, \dots, y_n)$, indicating that the value of the attribute x equals to $f(y_1, y_2, \dots, y_n)$.
- It also implies dependencies among attributes.

- **Example**

Production	Semantic Rules
$E \rightarrow E^1 + T$	$E.value = E^1.value + T.value$
$E \rightarrow T$	$E.value = T.value$
$T \rightarrow T^1 * F$	$T.value = T^1.value * F.value$
$T \rightarrow F$	$T.value = F.value$
$F \rightarrow (E)$	$F.value = E.value$
$F \rightarrow num$	$F.value = num.value$

Decorating Syntax Tree

- Semantics by attribute grammars
 - The semantics of a symbol is defined by its attributes.
 - For example, a person can be defined by a set of attributes, including id, gender, height, age, etc.
 - The semantics of a sentence is a syntax tree with attributes satisfying the evaluation rules.
- Example
 - Decorated syntax tree of $1 + 2 * 3$.

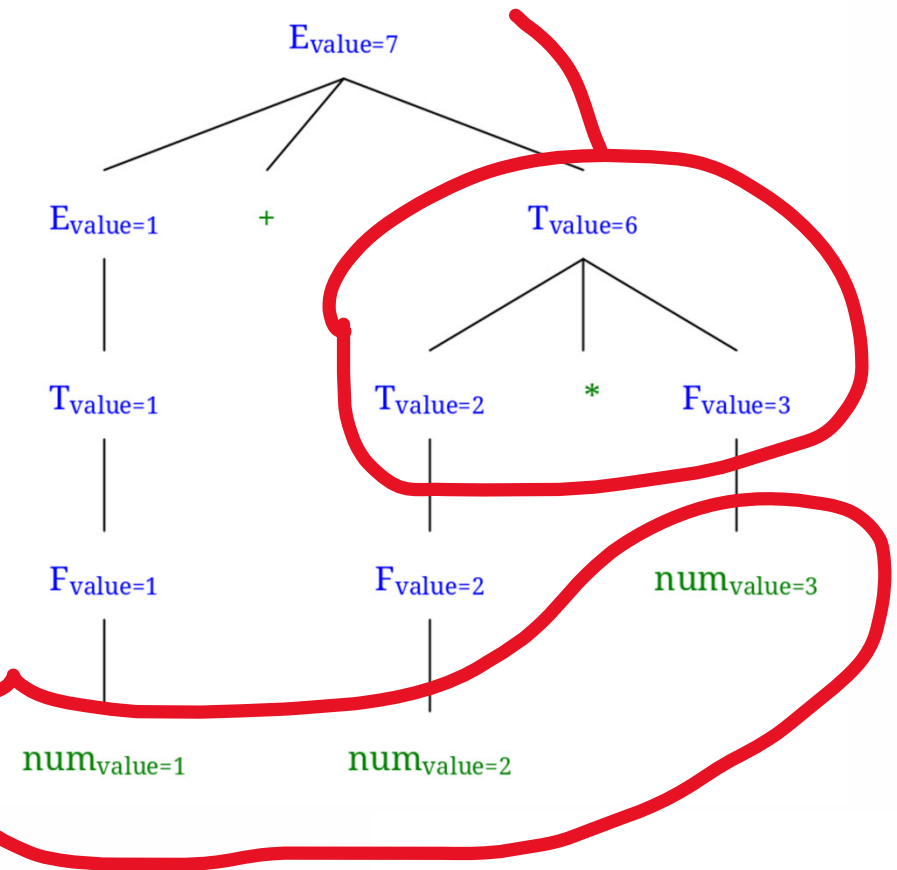


Decorating Syntax Tree

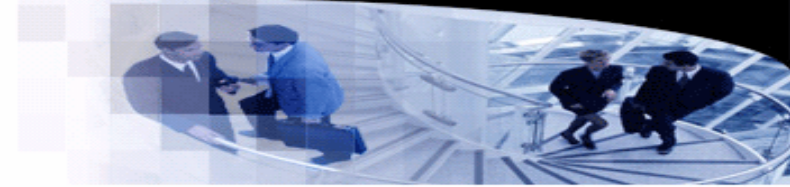
- Semantics by attribute grammars
 - The semantics of a symbol is defined by its attributes.
 - For example, a person can be defined by a set of attributes, including id, gender, height, age, etc.
 - The semantics of a sentence is a syntax tree with attributes satisfying the evaluation rules.
- Example
 - Decorated syntax tree of $1 + 2 * 3$.

For terminals, the attributes are normally assigned by the scanner.

$$T.value = T^1.value * F.value$$



Attribute Grammars

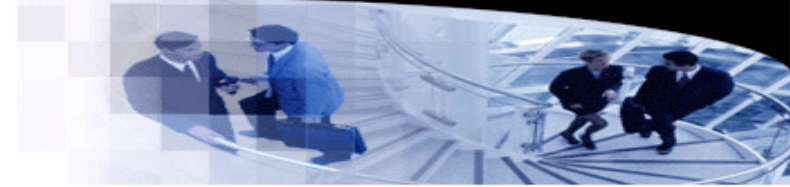


- Example

- Two's complement.
- Decorate the syntax tree of 1101.

Production	Semantic Rules
$N \rightarrow SD$	$N.val = \text{if } S.neg \text{ then } D.val - 2^{D.len} \text{ else } D.val$
$S \rightarrow bit$	$S.neg = \text{if } bit.val == 0 \text{ then false else true}$
$D \rightarrow D^1 bit$	$D.val = D^1.val * 2 + bit.val$ $D.len = D^1.len + 1$
$D \rightarrow bit$	$D.val = bit.val$ $D.len = 1$
$bit \rightarrow 0$	$bit.val = 0$
$bit \rightarrow 1$	$bit.val = 1$

Attribute Grammars

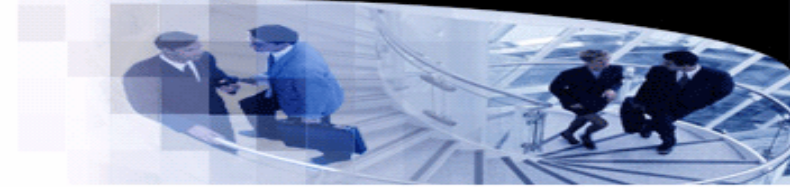


- Example

- Infix expressions to postfix expressions.
- Check the infix expression: $(a + b) * (c + d)$.

Production	Semantic Rules
$E \rightarrow E^1 + T$	$E.pst = E^1.pst + T.pst + " + "$
$E \rightarrow T$	$E.pst = T.pst$
$T \rightarrow T^1 * F$	$T.pst = T^1.pst + F.pst + " * "$
$T \rightarrow F$	$T.pst = F.pst$
$F \rightarrow (E)$	$F.pst = E.pst$
$F \rightarrow id$	$F.pst = id.text$

Attribute Grammars

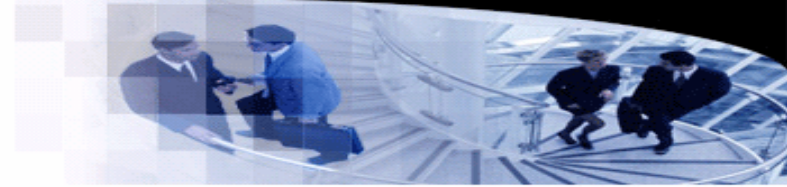


• Example

- Infix expressions to abstract syntax trees.
- Check the infix expression: $(a + b) * (c + d)$.

Production	Semantic Rules
$E \rightarrow E^1 + T$	$E.ast = node(+)$ $E.ast[0] = E^1.ast$ $E.ast[1] = T.ast$
$E \rightarrow T$	$E.ast = T.ast$
$T \rightarrow T^1 * F$	$T.ast = node(*)$ $T.ast[0] = T^1.ast$ $T.ast[1] = F.ast$
$T \rightarrow F$	$T.ast = F.ast$
$F \rightarrow (E)$	$F.ast = E.ast$
$F \rightarrow id$	$F.ast = node(id.text)$

Attribute Grammars



- Example

- C-style declarations.
- Check the declaration: `int p, *q.`

Production	Semantic Rules
$D \rightarrow SL$	$L.spec = S.spec$
$S \rightarrow int$	$S.spec = integer$
$L \rightarrow L^1, R$	$L^1.spec = L.spec$ $R.spec = L.spec$ $R.type = makeType(R.spec)$
$L \rightarrow R$	$R.spec = L.spec$ $R.type = makeType(R.spec)$
$R \rightarrow * R^1$	$R^1.spec = R.spec$ $R^1.type = pointer(R.type)$
$R \rightarrow id$	$id.type = R.type$